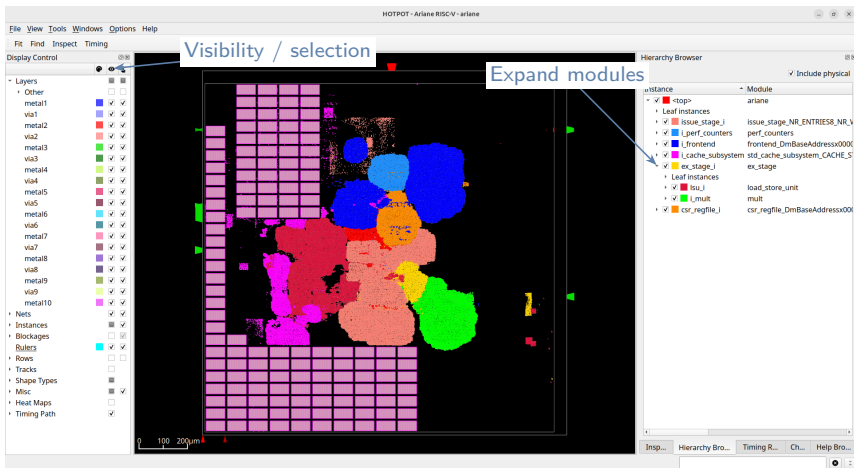
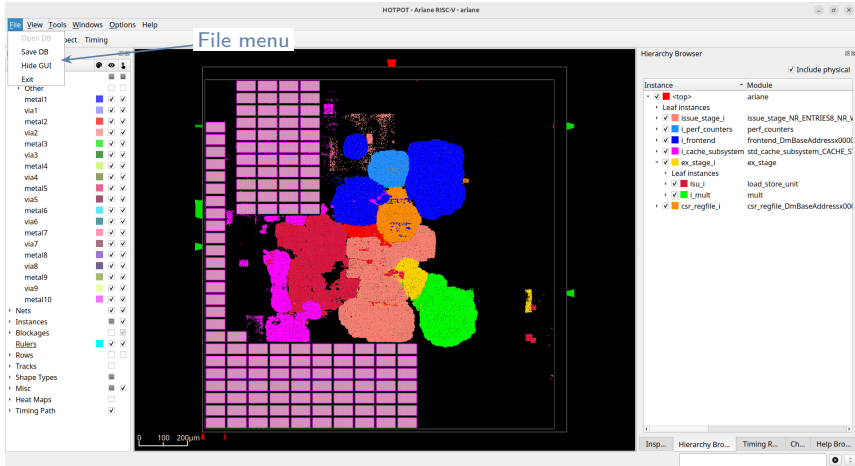


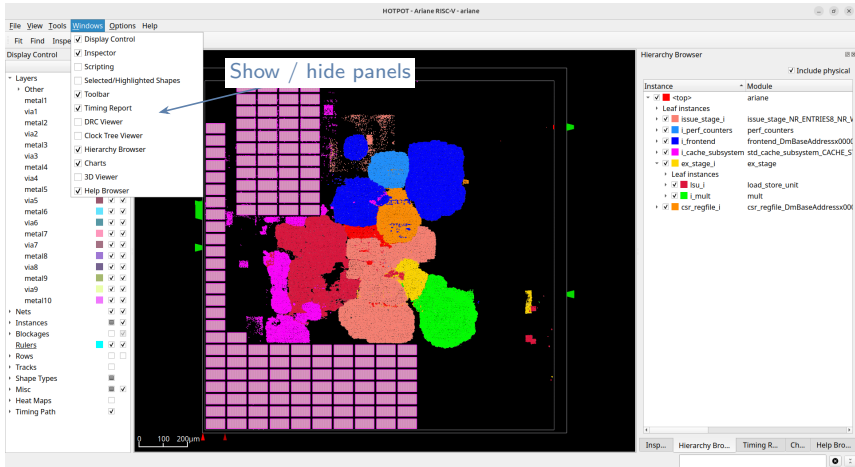
Explore Ariane: a public RISC-V core



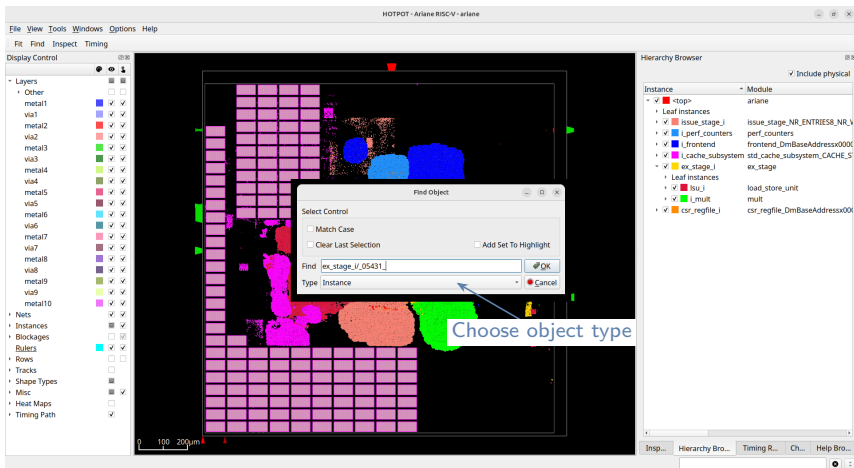
File: open and save databases



Windows: choose the panels



Find an instance by name



5 / 52



ITerm: follow the connections



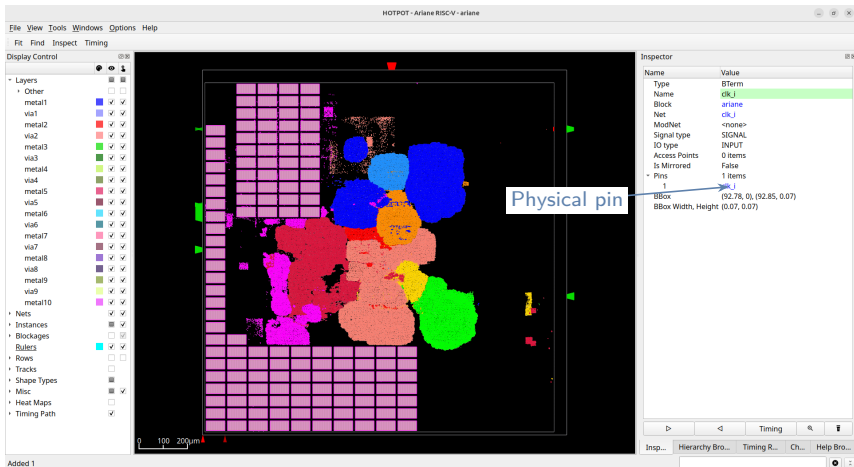
Net: inspect its endpoints

Click an endpoint

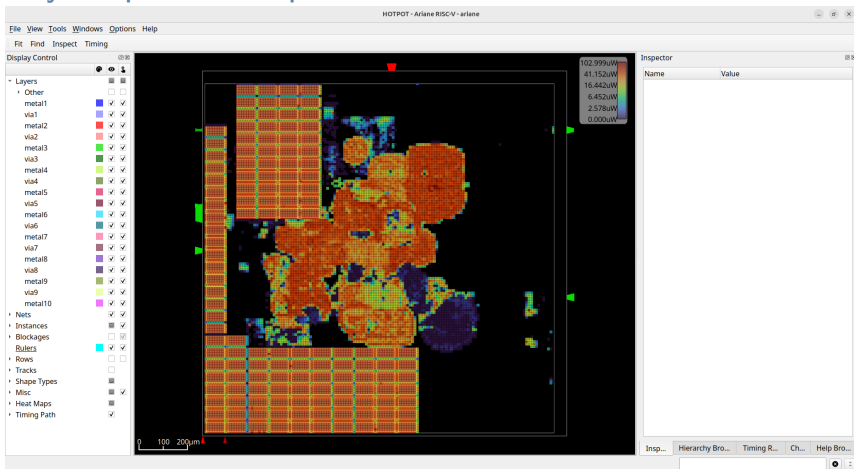
Inspector

Name	Value
Type	Net
Name	alu_valid_id_ex
Block	ariane
Signal type	SIGNAL
Source type	NONE
Wire type	ROUTED
Special	False
Don't Touch	False
I-Terms	2 Items
1	ex_stage_17_05431_/A
2	issue_stage_17_05431_/A
O-Terms	0 Items
Buffer tree	alu_valid_id_ex
BBox	(804.51, 639.985), (805.79, 641.05)
BBox Wldt...	(1.28, 1.065)

BTerm: a top-level port

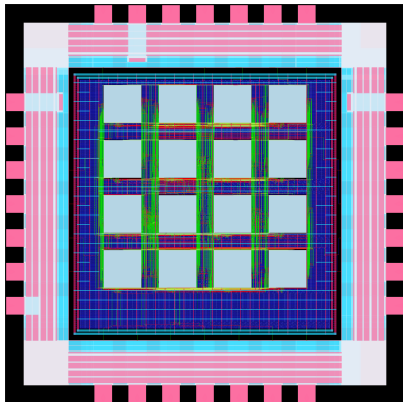


Power density: explore the spatial variation

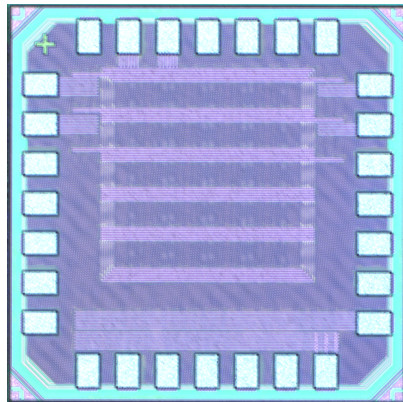


Estimated: 1 ns clock · 0.1 input transitions/cycle · 50% duty
Synthetic RAM models; no workload trace or thermal simulation

Frida: floorplan and fabricated silicon



IHP130 layout



Fabricated TSMC65 die

Similar architecture; different processes and geometries.

The command reference

Database
GUI
Web Viewer
Synthesis
Partition Manager
Restructure
Floorplan Initialization
Pin Placement
Design for Test
Chip-level Connections
Macro Placement
Tapcell Insertion
PDN Generation
Global Placement
Static Timing Analyzer
(external)
Parasitics Estimation
Gate Resizing
Clock gating
Detailed Placement



Options

Switch Name	Description
<code>-core_area</code>	Core area coordinates in microns. Either a rectangle as <code>{llx lly urx ury}</code> , or a rectilinear polygon as a list of vertex coordinates <code>{x1 y1 x2 y2 ...}</code> . See Rectilinear Floorplans .
<code>-core_space</code>	Space around the core in microns. Allowed values are either one value for all margins or a set of four values, one for each margin. The order of the four values are: <code>{bottom top left right}</code> .
<code>-die_area</code>	Die area coordinates in microns. Either a rectangle as <code>{llx lly urx ury}</code> , or a rectilinear polygon as a list of vertex coordinates <code>{x1 y1 x2 y2 ...}</code> . See Rectilinear Floorplans .
<code>-site</code>	Site name.
<code>-utilization</code>	Percentage utilization. Allowed values are <code>double</code> in the range <code>{0-100}</code> .
<code>[-additional_sites]</code>	Tcl list of sites to make rows for (e.g. <code>{SITEXX SITEYY}</code>)

Start the workshop script

```
export OPENROAD_ROOT=/path/to/OpenROAD  
openroad -gui flow.tcl
```

Run from the public repository root.

Read the physical library

```
read_lef $tech/Nangate45_tech.lef
read_lef $tech/Nangate45_stdcell.lef
Technology    Cell masters
```

Read the timing library

```
read_liberty $tech/Nangate45_typ.lib  
1.1 V      25 °C
```

Read and link the netlist

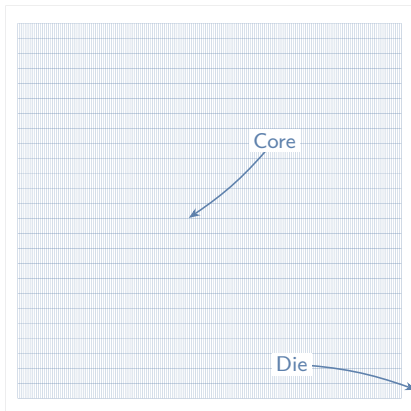
```
read_verilog $netlist
```

```
link_design gcd
```

Top module: gcd

Floorplan comes next

Initialize the floorplan



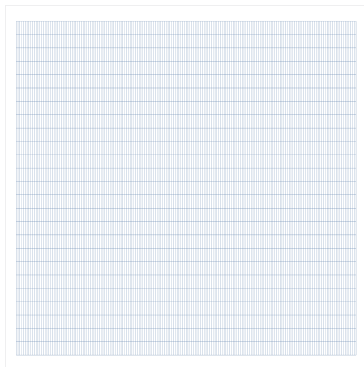
```
initialize_floorplan
  -site <site>
  -utilization <percent>
  -core_space <micrometres>
  [-aspect_ratio <ratio>]
```

Default: ratio 1.0

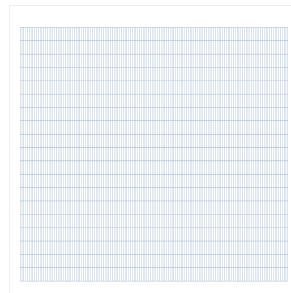
Shown: 40%, margin 1 μ m

[argument] optional
<value> placeholder

Utilization sets the available area



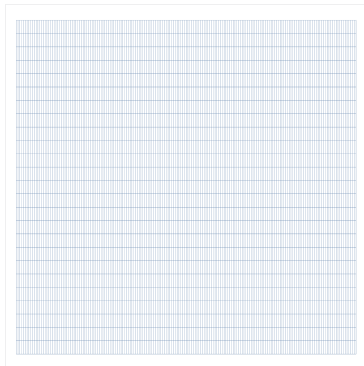
-utilization 40



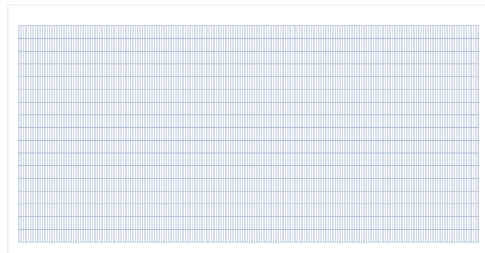
-utilization 65

$$A_{\text{core}} \approx A_{\text{cells}} / (u/100)$$

Choose the shape



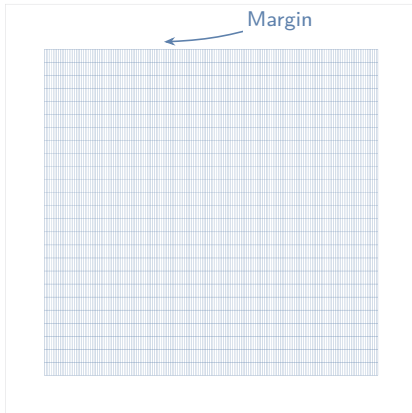
`-aspect_ratio 1`



`-aspect_ratio 0.5`

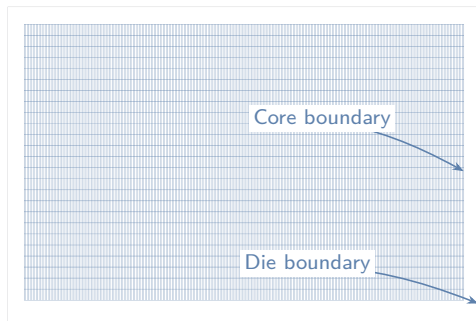
$$r = H/W \quad W = \sqrt{A/r} \quad H = \sqrt{Ar}$$

Reserve the core margin



```
initialize_floorplan  
-site $site  
-utilization 40  
-core_space 4
```

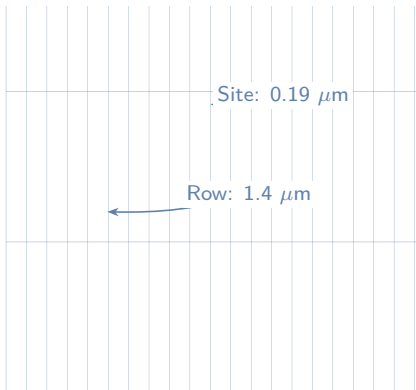
Use a prescribed footprint



```
initialize_floorplan  
-site $site  
-die_area {0 0 60 40}  
-core_area {2 2 58 38}
```

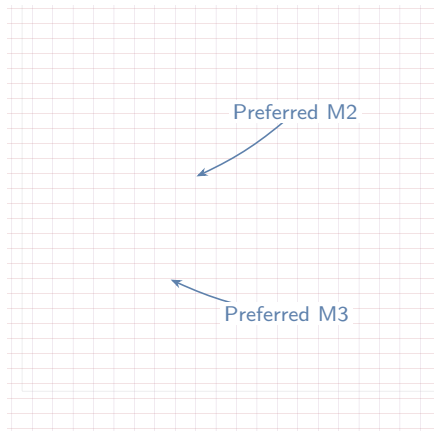
Coordinates in μm

Rows and sites



```
initialize_floorplan
-site $site
-utilization 40
-core_space 1
```

Create routing tracks



`make_tracks`

Pitch and offset from technology LEF

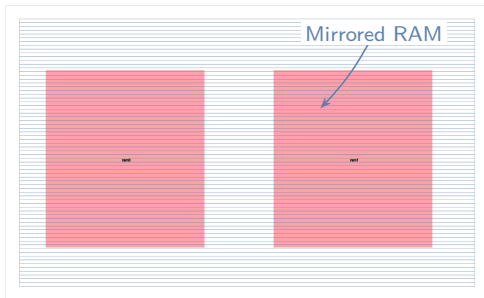
A separate macro example



```
openroad -gui <macros.tcl>
```

Fresh session: two RAM abstracts + GCD

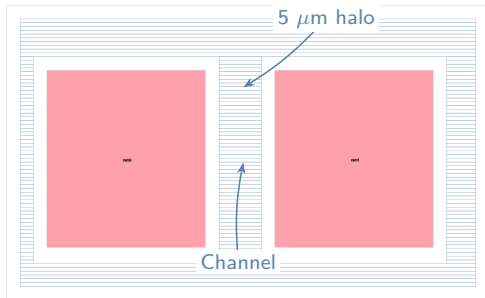
Place and orient a macro



```
place_macro  
  -macro_name ram1  
  -location {100 20}  
  [-orientation MY]
```

Default orientation: R0

Leave access around the macros



```
cut_rows  
[-halo_width_x 5]  
[-halo_width_y 5]
```

Automatic macro placement



`rtl_macro_placer`

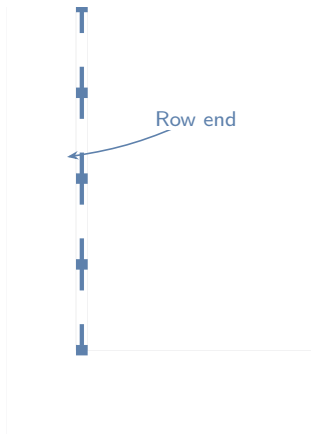
`macros_auto.tcl`: 5 μ m halos

Hierarchical clustering

Simulated annealing

Wirelength + outline + packing

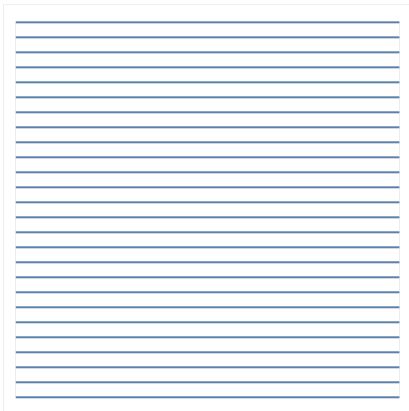
Insert taps and endcaps



```
tapcell  
  [-distance 120]  
  [-tapcell_master TAPCELL_X1]  
  [-endcap_master TAPCELL_X1]
```

GCD branch

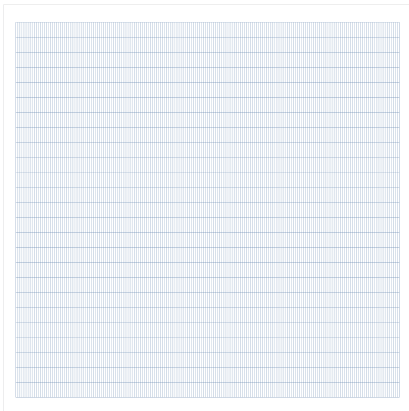
Connect the supply pins



```
add_global_connection  
-net VDD  
-pin_pattern {^VDD$}  
[-power]
```

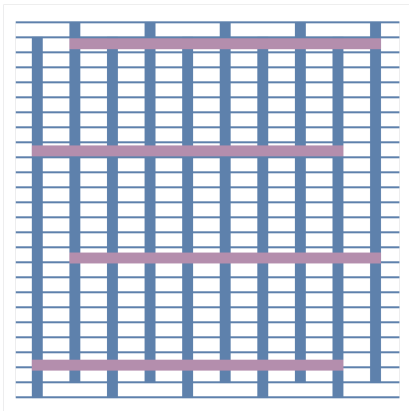
Repeat for VSS with -ground

Define the power domain



```
set_voltage_domain  
-name CORE  
-power VDD  
-ground VSS
```

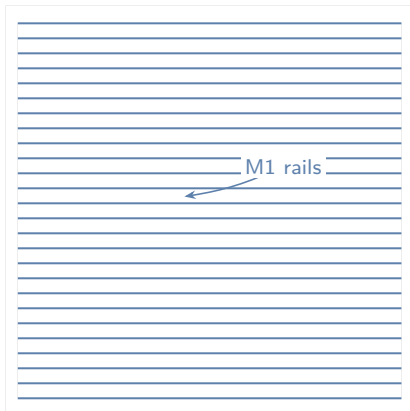
Define the grid



```
define_pdn_grid  
  [-name core_grid]  
  [-voltage_domains CORE]
```

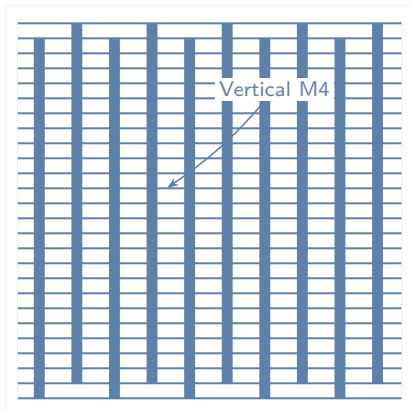
Grid policy; geometry at pdngen

Follow the row power rails



```
add_pdn_stripe  
[-grid core_grid]  
-layer metal1  
[-followpins]
```

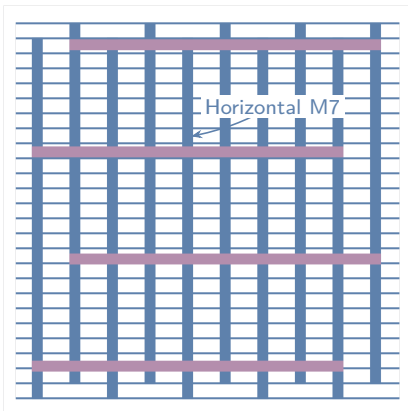
Choose strap width and pitch



```
add_pdn_stripe  
  [-grid core_grid]  
  -layer metal4  
  [-width 1]  
  [-pitch 7]  
  [-offset 2]
```

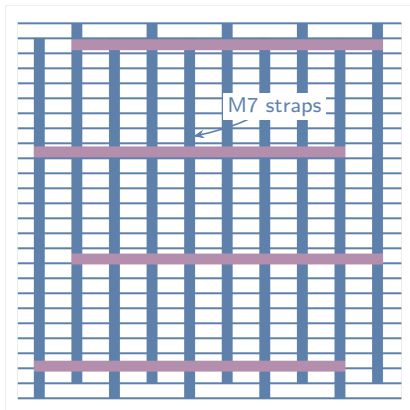
Pitch repeats a VDD/VSS pair

Add a crossing strap layer



```
add_pdn_stripe
-layer metal7
[-width 1]
[-pitch 20]
[-offset 3]
[-number_of_straps 2]
```

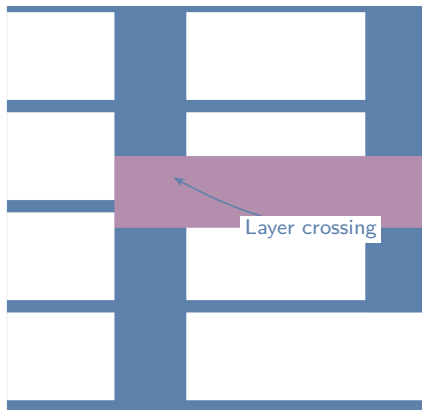
Connect the power layers



```
add_pdn_connect  
  [-grid core_grid]  
  -layers {metal1 metal4}
```

Also connect M4 to M7

Generate the power grid



pdngen

Rule-based straps and via insertion

When do we place the pins?

Pins first

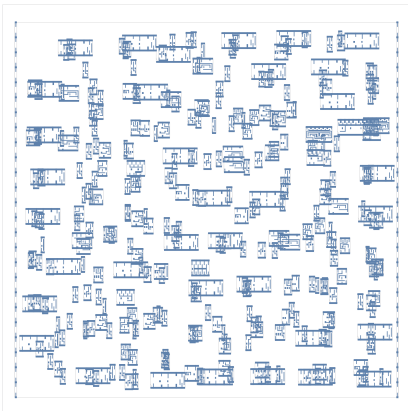
Pin constraints → pin locations → cell placement

Wirelength-driven pin locations

Rough cells → optimize pins → refine cells

Unplaced cell endpoints are modeled at the die center.

Optional: rough cells before pin optimization



```
global_placement  
  [-density 0.80]  
  [-pad_left 2]  
  [-pad_right 2]  
  [-skip_io]
```

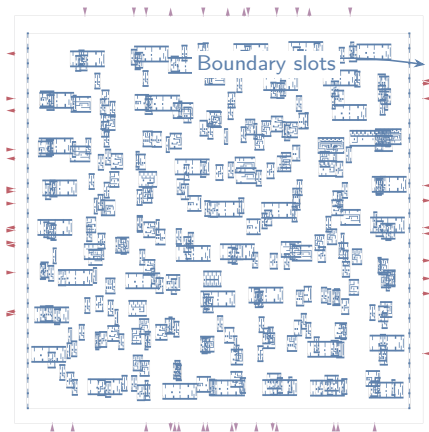
Rough global placement; detailed placement comes later

Smooth wirelength + electrostatic density

Nesterov optimization

Defaults: density 0.7; padding 0

Automatic pin placement



```
place_pins
```

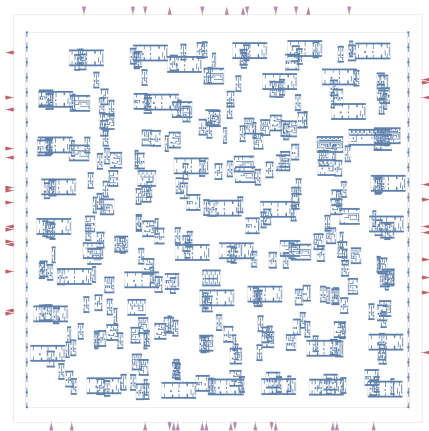
```
-hor_layers metal3
```

```
-ver_layers metal2
```

Fixed cell locations + track slots

Unplaced cells: modeled at die center

The pin-assignment objective



$$c_{ps} = \text{HPWL}(S_p \cup \{s\})$$

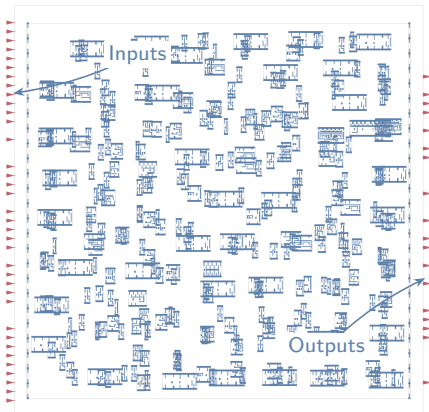
S_p : fixed cell-pin locations

Hungarian matching within slot sections

Optional simulated annealing

Geometric cost; no routed paths

Restrict pins to an edge

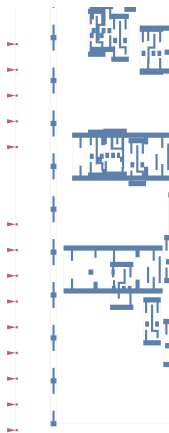


```
set_io_pin_constraint  
  [-direction input]  
  [-region left:*
```

Outputs: -region right:*

Then rerun place_pins

Control the minimum separation



```
place_pins
```

```
-hor_layers metal3
```

```
-ver_layers metal2
```

```
[-min_distance 0.8]
```

0.8 μm minimum; track snapped

-min_distance_in_tracks: track count

Default separation: two routing tracks

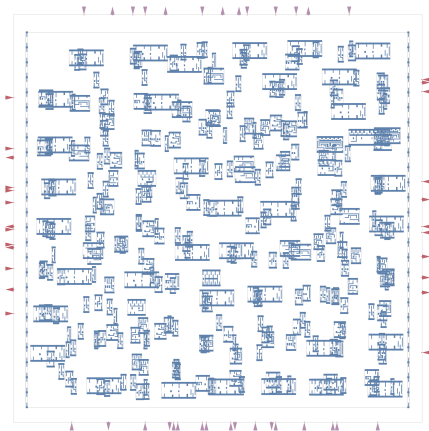
Keep a bus together



```
set_io_pin_constraint
[-pin_names {resp_msg[*]}]
[-region right:*)
[-group]
[-order]
```

Then rerun place_pins

Try the annealing solver

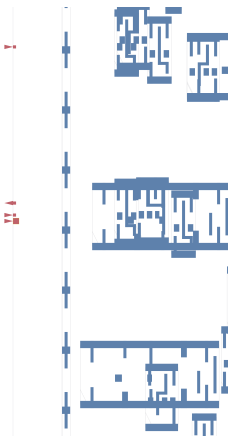


```
place_pins  
  -hor_layers metal3  
  -ver_layers metal2  
  [-annealing]
```

Same HPWL objective

Try edge + bus constraints

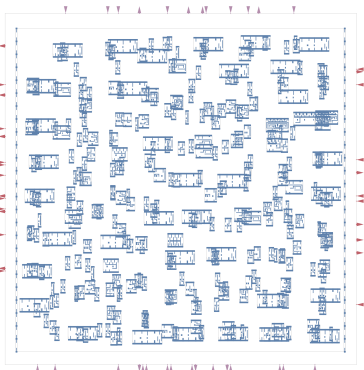
Fix one pin first



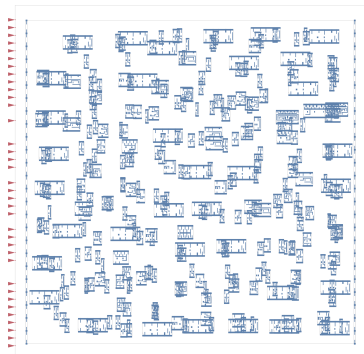
```
place_pin
  -pin_name clk
  -layer metal3
  -location {0 10}
  [-force_to_die_boundary]
```

Then place_pins for the remaining ports

Measure the cost of constraints



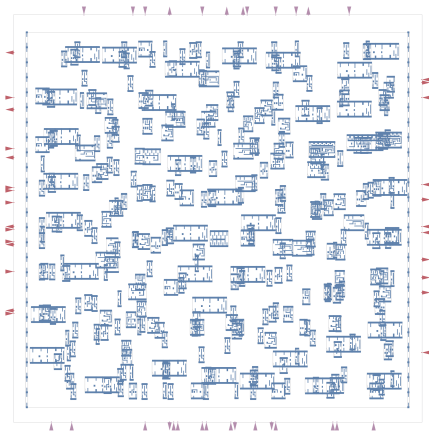
Automatic: 536.82 μm



Constrained: 1126.27 μm

Same fixed cells; constraints: opposite edges + 0.8 μm spacing

Refine cells with the pins included



```
global_placement  
  [-routability_driven]  
  [-density 0.80]  
  [-pad_left 2]  
  [-pad_right 2]
```

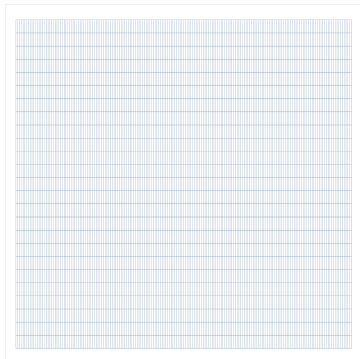
Pins now contribute to the placement objective

RUDY congestion estimate + cell inflation

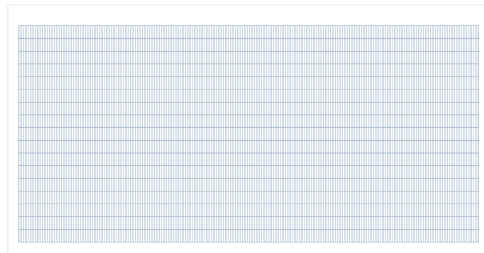
-routability_use_grt: FastRoute feedback

Defaults: density 0.7; padding 0

Exercise: compare two floorplans



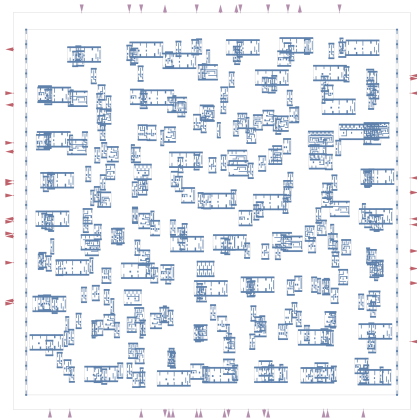
$$u = 40\%, H/W = 1$$



$$u = 40\%, H/W = 0.5$$

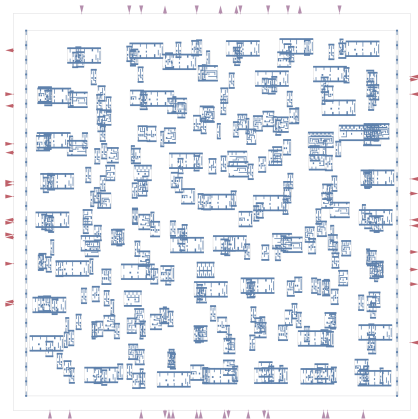
Compare I/O HPWL and pin constraints after placement

Save a half-finished design: ODB



```
write_db  
  partial.odb  
Physical database checkpoint
```

Resume the ODB in a fresh session



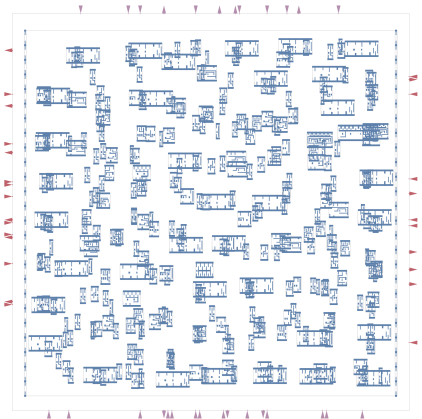
```
read_db
```

```
    partial.odb
```

Technology + cell masters are included

Timing: also read Liberty + SDC

Save a half-finished design: DEF

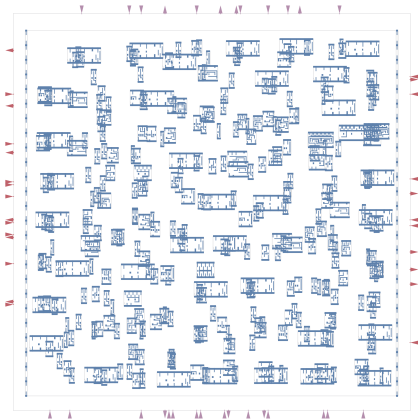


```
write_def
```

```
    partial.def
```

Current rows, pins and placement; routes if present

Read the DEF in a fresh session



```
read_lef $tech/Nangate45_tech.lef
read_lef $tech/Nangate45_stdcell.lef
```

```
read_def
    partial.def
```

LEFs supply technology + cell masters
Timing: also read Liberty + SDC

Open the workshop checkpoints

```
openroad -gui examples/floorplanning/read_odb.tcl
openroad -gui examples/floorplanning/read_def.tcl
```

Separate sessions; run from slides/.

```
cd examples/ariane
make run
make gui
```

Ariane: pinned downloads + standalone OpenROAD.